

History of JavaScript

Did you know that initially, JavaScript was developed in just 10 days? Yes, you heard it right. JavaScript was created by **Brendan Eich**, a programmer at a company called Netscape Communications.

Originally, JavaScript was named '**Mocha**', but it quickly gained popularity as '**LiveScript**' and later evolved into '**JavaScript**'.

Many JavaScript frameworks and libraries, such as AngularJS, ReactJS, and Vue.js, have revolutionized frontend development. Additionally, Node.js, a powerful JavaScript runtime environment, has empowered developers to build server-side applications using JavaScript.

JavaScript : A Programming Language or a Scripting Language?

Scripting Languages

Scripting languages are those languages which are interpreted rather than compiled, meaning they are executed line-by-line by the interpreter at the runtime thus allowing for quicker development cycles and easier debugging.

Scripting languages are commonly used for tasks such as system administration, web development, text processing and automation.

JavaScript is widely used for client-side scripting in web browsers, it has become a versatile language for web development including front-end and server side scripting with platforms like Node.js. Scripting languages are interpreted within a host environment; for instance, JavaScript needs to be integrated into HTML and then interpreted by the web browser.

Programming Languages

Programming languages are compiled languages, which means the source code is compiled to convert it to machine code.

A programming language is a computer language which is used to communicate with computers using a set of instructions. Programming languages utilize compilers and do not rely on interpretation by another language or application.

JavaScript : Is it a Client-Side or Server-Side Language?

For years, JavaScript has been predominantly associated with client-side scripting, breathing life into web pages and making them interactive.

JavaScript's roots lie firmly in the client-side realm. It's the language that adds the sparkle to your web experience, making elements appear and disappear, reacting to your clicks and scrolls, and seamlessly handling forms and animations.

In recent years, JavaScript has also gained prominence as a server-side language, thanks to the introduction of Node.js. Node.js is an open-source, JavaScript runtime environment that allows developers to run JavaScript code server-side. With Node.js, JavaScript can be used to build scalable, high-performance web applications, APIs, and backend services.

JavaScript Engines for Executing JavaScript Code

A JavaScript engine is the software that takes our JavaScript code and translates it into instructions that your computer can understand. It is like a powerful interpreter.

Some of the most popular JS engines today are :

1 : V8 - Developed by Google, V8 is one of the most widely used JavaScript engines. It powers the JavaScript execution in Google Chrome and Node.js.

2 : SpiderMonkey - Developed by Mozilla Firefox, SpiderMonkey is the JavaScript engine used in the Firefox web browser. It was one of the first JavaScript engines ever created and is known for its performance and compliance with JavaScript standards.

3 : JavaScriptCore - Used in Safari, known for its tight integration with Apple devices.

4 : Chakra - Developed by Microsoft, Chakra was the JavaScript engine used in the Edge web browser. It has now been replaced by the Chromium-based Edge browser, which uses V8 as its JavaScript engine.

JavaScript has 8 Datatypes

A JavaScript variable can hold 8 types of data:

Type	Description
String	A text of characters enclosed in quotes
Number	A number representing a mathematical value
Bigint	A number representing a large integer
Boolean	A data type representing true or false

Object	A collection of key-value pairs of data
Undefined	A primitive variable with no assigned value
Null	A primitive value representing object absence
Symbol	A unique and primitive identifier

JavaScript Data Types

A **Data Type** defines the type of data that a variable can store.

Example

```
let name = "Vicky";  
let age = 25;
```

Here:

- "Vicky" is a **String**
- 25 is a **Number**

Types of Data Types in JavaScript

JavaScript data types are divided into **two categories**:

1. Primitive Data Types

These are basic built-in data types:

1. String
2. Number
3. Boolean

4. Undefined
 5. Null
 6. BigInt
 7. Symbol
-

2. Non-Primitive (Reference) Data Types

1. Object
 2. Array
 3. Function
-

1. String Data Type

A **String** is used to store text.

```
let name = "Vicky";
```

```
console.log(name);
```

Output

Vicky

More Examples

```
let city = "Delhi";  
let college = 'ABC College';
```

```
console.log(city);  
console.log(college);
```

2. Number Data Type

A **Number** is used to store both integers and decimal values.

```
let age = 25;  
let salary = 45000.50;
```

```
console.log(age);  
console.log(salary);
```

Output

```
25  
45000.5
```

3. Boolean Data Type

A **Boolean** can have only two values:

- true
- false

```
let isStudent = true;  
let isMarried = false;
```

```
console.log(isStudent);  
console.log(isMarried);
```

Output

```
true  
false
```

4. Undefined Data Type

When a variable is declared but no value is assigned, it becomes **undefined**.

```
let x;
```

```
console.log(x);
```

Output

undefined

5. Null Data Type

Null represents an intentional absence of a value.

```
let data = null;
```

```
console.log(data);
```

Output

null

Difference Between Undefined and Null

Undefined	Null
Value has not been assigned	Value is intentionally empty
Assigned automatically by JavaScript	Assigned manually by the programmer

```
let a;
```

```
let b = null;
```

```
console.log(a);  
console.log(b);
```

6. BigInt Data Type

BigInt is used to store very large integers beyond the safe limit of the Number type.

```
let bigNumber = 123456789012345678901234567890n;  
  
console.log(bigNumber);
```

Output

```
123456789012345678901234567890n
```

7. Symbol Data Type

A **Symbol** creates a unique identifier.

```
let id = Symbol("101");  
  
console.log(id);
```

Output

```
Symbol(101)
```

8. Object Data Type

An **Object** stores multiple values in key-value pairs.


```
let student = {  
  name: "Vicky",  
  age: 25,  
  city: "Delhi"  
};  
  
console.log(student);
```

Output

```
{  
  name: "Vicky",  
  age: 25,  
  city: "Delhi"  
}
```

9. Array Data Type

An **Array** stores multiple values in a single variable.

```
let subjects = [  
  "HTML",  
  "CSS",  
  "JavaScript",  
  "Java"  
];  
  
console.log(subjects);
```

Output

```
["HTML", "CSS", "JavaScript", "Java"]
```

10. Function Data Type

A **Function** is a block of code designed to perform a specific task.

```
function greet() {  
  console.log("Welcome");  
}
```

```
greet();
```

Output

Welcome

typeof Operator

The **typeof** operator is used to determine the data type of a variable.

```
let name = "Vicky";  
let age = 25;  
let status = true;
```

```
console.log(typeof name);  
console.log(typeof age);  
console.log(typeof status);
```

Output

```
string  
number  
boolean
```

JavaScript Numbers

All JavaScript numbers are stored as decimal numbers (floating point).

Example

```
let x1 = 34.00;
```

```
let x2 = 34;
```

Exponential Notation

```
let y = 123e5; // 12300000
```

```
let z = 123e-5; // 0.00123
```

Datatype undefined

A variable without a value has the datatype **undefined**.

Let, Var & Const

JavaScript variables can be declared in 4 ways:

Modern JavaScript

- Using let
- Using const

Older JavaScript

- Using var (Not Recommended)
- Automatically (Not Recommended)

Example using let

```
let x = 5;
```

```
let y = 6;
```

```
let z = x + y;
```

Example using const

```
const x = 5;
```

```
const y = 6;
```

```
const z = x + y;
```

JavaScript Identifiers

Variables are identified with unique names called identifiers.

Names can be short like x, y, z.

Names can be descriptive age, sum, carName.

The rules for constructing names (identifiers) are:

Names can contain letters, digits, underscores, and dollar signs.

Names must begin with a letter, a \$ sign or an underscore (_).

Names are case sensitive (X is different from x).

Reserved words (JavaScript keywords) cannot be used as names.

JavaScript Underscore (_)

JavaScript treats underscore as a letter.

Identifiers containing _ are valid variable names:

Example

```
let _lastName = "Johnson";
```

```
let _x = 2;
```

```
let _100 = 5;
```

A convention among professional programmers is to start a name with underscore for "private" variables.

JavaScript Dollar Sign \$

JavaScript also treats a dollar sign as a letter.

Identifiers containing \$ are valid variable names:

Example

```
let $ = "Hello World";
```

```
let $$$ = 2;
```

```
let $myMoney = 5;
```

Using the \$ is not very common in JavaScript, but professional programmers often use it as an alias for the main function in JavaScript libraries.

var

The var keyword was the traditional way of declaring variables in JavaScript. Variables declared with **var are function-scoped or globally-scoped, but not block-scoped**. This means that variables declared with var are hoisted to the top of their function or global scope. This hoisting behaviour can sometimes lead to unexpected results and make debugging challenging.

var is function scoped

When we declare a variable using var inside a function, we cannot access the variable outside the function.

```
function example() {
```

```
    var x = 10;
```

```
    console.log(x);
```

```
}
```

```
example();    // 10
```

```
console.log(x);    // Throws ReferenceError: x is not defined
```

Reassignment and Redefinition using var

In JavaScript, the var keyword allows variables to be both reassigned and redefined within the same scope, unlike let which only allows reassignment. This behavior of var can sometimes lead to unexpected results and bugs in code if not used carefully.

// Re assignment using var

```
var x = 10;
console.log(x); // Output : 10
x = "Mayank";
console.log(x); // Output : Mayank
// Re definition using var
var x = 10;
console.log(x); // Output : 10
var x = "Mayank";
console.log(x); // Output : Mayank
```

var has some quirks and drawbacks, especially related to variable hoisting and its lack of block scope. Due to these issues, let and const were introduced in ES6 (ECMAScript 2015) to provide better variable scoping mechanisms. It's generally recommended to use let and const instead of var in modern JavaScript code, unless there's a specific reason to use var.

JavaScript Let

- The let keyword was introduced in [ES6 \(2015\)](#)
- Variables declared with let have Block Scope
- Variables declared with let must be Declared before use
- Variables declared with let cannot be Redeclared in the same scope

Block Scope

- Before ES6 (2015), JavaScript did not have Block Scope.
- JavaScript had Global Scope and Function Scope.
- ES6 introduced the two new JavaScript keywords: let and const.

These two keywords provided Block Scope in JavaScript:

Example

Variables declared inside a { } block cannot be accessed from outside the block:

```
{  
  let x = 2;  
}  
// x can NOT be used here
```

Global Scope

Variables declared with the var always have Global Scope.

Variables declared with the var keyword can NOT have block scope:

Example

Variables declared with var inside a { } block can be accessed from outside the block:

```
{  
  var x = 2;  
}  
// x CAN be used here
```

Cannot be Redeclared

Variables defined with let can not be redeclared.

```
let x = "John Doe";
```

```
let x = 0;
```

Variables defined with var can be redeclared.

```
var x = "John Doe";
```

```
var x = 0;
```

Redeclaring Variables

Redeclaring a variable using the var keyword can impose problems.

Redeclaring a variable inside a block will also redeclare the variable outside the block:

Example

```
var x = 10;
// Here x is 10
{
  var x = 2;
  // Here x is 2
}
// Here x is 2
```

Redeclaring a variable using the let keyword can solve this problem. Redeclaring a variable inside a block will not redeclare the variable outside the block:

Example

```
let x = 10;
// Here x is 10
{
  let x = 2;
  // Here x is 2
}
// Here x is 10
```

Difference Between var, let and const

	Scope	Redeclare	Reassign	Hoisted	Binds this
Var	No	Yes	Yes	Yes	Yes
Let	Yes	No	Yes	No	No
Const	Yes	No	No	No	No

Let Hoisting

Variables defined with var are hoisted to the top and can be initialized at any time.

Meaning: You can use the variable before it is declared:

```
carName = "Volvo";  
var carName;
```

In JavaScript, the **let keyword is used to declare variables that are block-scoped**, meaning they are only accessible within the block in which they are defined. Block scope refers to any code block delimited by curly braces {} such as loops, conditionals, or function bodies.

```
let x = 10;  
if (true) {  
    let y = 20;  
    console.log(x); // Output: 10  
    console.log(y); // Output: 20  
}  
console.log(x); // Output: 10  
console.log(y); // Throws ReferenceError: y is not defined
```

Reassignment and Redefinition using let

Variables declared with let can be reassigned, just like those declared with var, but they cannot be redeclared within the same block scope. This helps prevent accidental redeclaration of variables and can aid in writing more predictable and maintainable code.

// Reassignment using 'let'

```
let x = 10;
console.log(x); // Output : 10
x = "Mayank";
console.log(x); // Output : Mayank
// Redefinition using 'let'
let x = 10;
let x = "Mayank";
console.log(x); // Output : Error!
```

const

In JavaScript, the `const` keyword is used to declare constants. Constants are variables whose values cannot be reassigned once they are initialized. This means that once a value is assigned to a constant using `const`, it cannot be changed or reassigned throughout the execution of the script.

```
const x = 20;
console.log(x); // Output : 20
x = 30;
console.log(x); // Output : Error!
```

Constants declared with `const` must be initialized with a value. Unlike variables declared with `var` or `let`, which can be declared without immediately assigning a value, `const` requires initialization at the time of declaration.

```
let x; // No Error
var y; // No Error
const z; // Error! "Missing initializer in const declaration"
```

It's important to note that while the value assigned to a constant cannot be changed, if the value is an object or an array, its properties or elements can still be modified. **This is because const only prevents reassignment of the variable itself, not its properties or elements.**

```
const person = {  
  name: 'Mayank',  
  age: 21  
};  
console.log(person.name); // Output: Mayank  
// Modifying object properties is allowed  
person.name = 'Love';  
console.log(person.name); // Output: Love  
// Attempting to reassign the variable will throw an error  
person = {}; // Throws TypeError: Assignment to constant variable.  
Similarly with arrays...  
const numbers = [1, 2, 3];  
console.log(numbers); // Output: [1, 2, 3]  
// Modifying array elements is allowed  
numbers.push(4);  
console.log(numbers); // Output: [1, 2, 3, 4]  
// Attempting to reassign the variable will throw an error  
numbers = []; // Throws TypeError: Assignment to constant variable.  
It is important to remember that for complex data types like objects  
and arrays, const only prevents reassignment of the variable itself,  
not its properties or elements.
```

Variable/Function naming conventions

a) Camel Case

Phrase	Camel case
Phone number	phoneNumber
Calculate interest rate	calculateInterestRate
Code help student	codeHelpStudent
Total sum	totalSum

b) Pascal Case

Phrase	Pascal case
Phone number	PhoneNumber
Calculate interest rate	CalculateInterestRate
Code help student	CodeHelpStudent
Total sum	TotalSum

c) Snake Case

Phrase	Snake case
Phone number	phone_number
Calculate interest rate	calculate_interest_rate
Code help student	code_help_student
Total sum	total_sum

d) Kebab Case

Phrase	Kebab case
Phone number	phone-number
Calculate interest rate	calculate-interest-rate
Code help student	code-help-student
Total sum	total-sum

Types of Operators in JavaScript

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. Increment & Decrement Operators
6. String Operators
7. Ternary Operator
8. Type Operators

1. Arithmetic Operators

Used to perform mathematical calculations.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Exponentiation

2. Assignment Operators

Used to assign values.

Operator	Example
=	x = 10
+=	x += 5
-=	x -= 5
*=	x *= 5
/=	x /= 5

3. Comparison Operators

Used to compare values.

Operator	Meaning
==	Equal
===	Equal Value and Type
!=	Not Equal
>	Greater Than
<	Less Than
>=	Greater Than or Equal
<=	Less Than or Equal

Difference Between == and ===

```
let a = 10;  
let b = "10";
```

```
console.log(a == b);  
console.log(a === b);
```

Output

true
false

Why?

== → Checks Value Only
=== → Checks Value + Data Type

4. Logical Operators

Used to combine conditions.

Operator	Meaning
&&	AND
!	NOT
	OR

5. Increment and Decrement Operators

Increment (++)

```
let x = 5;
```

```
x++;
```

```
console.log(x);
```

Output:

6

Decrement (--)

```
let x = 5;
```

```
x--;
```

```
console.log(x);
```

Output:

4

Pre Increment

```
let x = 5;
```

```
console.log(++x);
```

Output

6

Post Increment

```
let x = 5;
```

```
console.log(x++);
```

```
console.log(x);
```

Output

5

6

6. String Operator

The + operator joins strings.

Complete Program

```
<!DOCTYPE html>
<html>
<body>

<script>

let firstName = "Vicky";
let lastName = "Ji";
let fullName = firstName + " " + lastName;
document.write(fullName);
</script>
</body>
</html>
```

Output

Vicky Ji

7. Ternary Operator

Short form of if-else.

Syntax

condition ? true_value : false_value

Complete Program

```
<!DOCTYPE html>
<html>
<body>

<script>

let age = 20;

let result =
age >= 18 ? "Eligible for Voting" : "Not Eligible";

document.write(result);

</script>

</body>
</html>
```

Output

Eligible for Voting

8. Type Operators

typeof

Used to find the data type.

Complete Program

```
<!DOCTYPE html>
<html>
<body>
```

```
<script>
```

```
let name = "Vicky";
```

```
let age = 25;
```

```
let status = true;
```

```
document.write(typeof name);
```

```
document.write("<br>");
```

```
document.write(typeof age);
```

```
document.write("<br>");
```

```
document.write(typeof status);
```

```
</script>
```

```
</body>
```

```
</html>
```

Output

string

number

boolean

The ??= Operator in JavaScript

The **Nullish Coalescing Assignment Operator (??=)** was introduced in ES2021.

It assigns a value to a variable **only if the variable is null or undefined**.

Syntax

variable ??= value;

Equivalent to:

```
if(variable === null || variable === undefined)
{
    variable = value;
}
```

Example 1: Variable is Undefined

```
let name;
```

```
name ??= "Vicky";
```

```
console.log(name);
```

Output

Vicky

Since name is undefined, the value "Vicky" is assigned.

Example 2: Variable Already Has a Value

```
let name = "Rahul";
```

```
name ??= "Vicky";
```

```
console.log(name);
```

Output

Rahul

Since name is neither null nor undefined, no assignment occurs.

Example 3: Variable is Null

```
let city = null;
```

```
city ??= "Delhi";
```

```
console.log(city);
```

Output

Delhi

Since city is null, the new value is assigned.

Example 4: Empty String

```
let name = "";
```

```
name ??= "Guest";
```

```
console.log(name);
```

Output

(Empty string remains unchanged)

Why?

Because:

""

is **not null** and **not undefined**.

Example 5: Number Zero

```
let marks = 0;
```

```
marks ??= 100;
```

```
console.log(marks);
```

Output

0

0 is not null or undefined, so no assignment happens.

Difference Between ||= and ??=

Using ||=

```
let marks = 0;
```

```
marks ||= 100;
```

```
console.log(marks);
```

Output

100

Because 0 is considered **falsy**.

Using ??=

```
let marks = 0;
```

```
marks ??= 100;
```

```
console.log(marks);
```

Output

0

Because 0 is not null or undefined.

Comparison Table

Value	= Assigns?	??= Assigns?
undefined	Yes	Yes
null	Yes	Yes
0	Yes	No

false	Yes	No
""	Yes	No

Practical Example

Suppose a user doesn't enter a username.

```
let username = null;
```

```
username ??= "Guest User";
```

```
console.log(username);
```

Output

Guest User

Complete Program

```
<!DOCTYPE html>
<html>
<head>
  <title>??= Operator Example</title>
</head>
<body>

<script>

let userName;
let city = "Jaipur";
let marks = 0;
```



```
userName ??= "Guest";  
city ??= "Delhi";  
marks ??= 100;
```

```
document.write("User Name: " + userName + "<br>");  
document.write("City: " + city + "<br>");  
document.write("Marks: " + marks);
```

```
</script>
```

```
</body>
```

```
</html>
```

Output

User Name: Guest

City: Jaipur

Marks: 0

What is \$ (Dollar Sign) in Programming?

The **\$ (Dollar Sign)** is **not a special JavaScript keyword**. It is simply a **valid character for variable names, function names, and object names**.

JavaScript allows variable names to start with:

- Letters (a-z, A-Z)
- Underscore (_)
- Dollar Sign (\$)

1. Using \$ as a Variable Name

```
let $name = "Vicky";
```

```
console.log($name);
```

Output

Vicky

2. Using \$ in Variable Names

```
let $price = 500;
```

```
let total$amount = 1000;
```

```
console.log($price);
```

```
console.log(total$amount);
```

Output

500

1000

3. Using Only \$

```
let $ = "Hello JavaScript";
```

```
console.log($);
```

Output

Hello JavaScript

Although valid, this is generally not recommended.

4. Template Literals (Most Common Use)

Many beginners think \$ has special meaning everywhere, but in JavaScript its most famous use is inside **Template Literals**.

```
let name = "Vicky";  
let age = 25;  
  
console.log(`My name is ${name}`);  
console.log(`My age is ${age}`);
```

Output

```
My name is Vicky  
My age is 25
```

Without \${}

```
let name = "Vicky";  
  
console.log("My name is " + name);
```

Output

```
My name is Vicky
```

With \${}

```
let name = "Vicky";  
  
console.log(`My name is ${name}`);
```

This is cleaner and easier to read.

5. Expressions Inside \${}

```
let a = 10;
```

```
let b = 20;
```

```
console.log(`Sum = ${a+b}`);
```

Output

Sum = 30

6. Multiple Variables

```
let name = "Vicky";
```

```
let city = "Jaipur";
```

```
console.log(`${name} lives in ${city}`);
```

Output

Vicky lives in Jaipur

7. HTML Generation (Real Web Development)

```
<script>
```

```
let product = "Laptop";
```

```
let price = 50000;
```

```
let card = `
```

```
<div>
```

```
<h2>${product}</h2>
<p>Price: ₹${price}</p>
</div>
`;
```

```
document.write(card);
```

```
</script>
```

Output

Laptop

Price: ₹50000

This technique is heavily used in React, Angular, Vue, and modern web applications.

8. Generate Dynamic Cards Using Loop

```
<script>
```

```
let products = ["Laptop","Mobile","Tablet"];
```

```
for(let i=0;i<products.length;i++)
{
  document.write(`
    <h3>${products[i]}</h3>
  `);
}
```

```
</script>
```

Output

Laptop

Mobile

Tablet

What is the Spread Operator (...) in JavaScript?

The **Spread Operator (...)** is used to **expand (spread)** the elements of an array, object, or string into individual values.

It was introduced in **ES6 (ECMAScript 2015)**.

Syntax

...

1. Spread Operator with Arrays

Without Spread Operator

```
let arr1 = [10, 20, 30];  
let arr2 = arr1;
```

```
console.log(arr2);
```

Output:

```
[10, 20, 30]
```

With Spread Operator

```
let arr1 = [10, 20, 30];
```

```
let arr2 = [...arr1];
```

```
console.log(arr2);
```

Output:

[10, 20, 30]

Here, all elements of arr1 are copied into arr2.

2. Merging Arrays

Traditional Method

```
let arr1 = [10,20];
```

```
let arr2 = [30,40];
```

```
let arr3 = arr1.concat(arr2);
```

```
console.log(arr3);
```

Output:

[10,20,30,40]

Using Spread Operator

```
let arr1 = [10,20];
```

```
let arr2 = [30,40];
```

```
let arr3 = [...arr1, ...arr2];
```

```
console.log(arr3);
```

Output:

[10,20,30,40]

3. Add New Elements

```
let numbers = [10,20,30];
```

```
let newNumbers = [...numbers, 40, 50];
```

```
console.log(newNumbers);
```

Output:

```
[10,20,30,40,50]
```

4. Add Elements at Beginning

```
let numbers = [30,40,50];
```

```
let newNumbers = [10,20,...numbers];
```

```
console.log(newNumbers);
```

Output:

```
[10,20,30,40,50]
```

5. Spread Operator with Strings

A string can be spread into individual characters.

```
let name = "Vicky";
```

```
console.log([...name]);
```

Output:

```
["V","i","c","k","y"]
```

6. Spread Operator with Objects

```
let student =  
{  
  name:"Vicky",  
  age:25  
};
```

```
let newStudent =  
{  
  ...student  
};
```

```
console.log(newStudent);
```

Output:

```
{  
  name:"Vicky",  
  age:25  
}
```

7. Merge Objects

```
let personal =  
{  
  name:"Vicky"  
};
```

```
let professional =
```

```
{  
  profession:"Professor"  
};
```

```
let details =  
{  
  ...personal,  
  ...professional  
};
```

```
console.log(details);
```

Output:

```
{  
  name:"Vicky",  
  profession:"Professor"  
}
```

8. Override Object Properties

```
let student =  
{  
  name:"Vicky",  
  age:25  
};
```

```
let updatedStudent =  
{  
  ...student,  
  age:30  
};
```

```
console.log(updatedStudent);
```

Output:

```
{  
  name:"Vicky",  
  age:30  
}
```

The last value overwrites the previous one.

9. Function Arguments

```
function add(a,b,c)  
{  
  return a+b+c;  
}
```

```
let nums = [10,20,30];
```

```
console.log(add(...nums));
```

Output:

60

Without spread:

```
add(nums);
```

This would not work correctly because the function expects three separate arguments.

10. Find Maximum Number

```
let numbers = [5,10,25,15];  
  
console.log(Math.max(...numbers));
```

Output:

25

Real-World Example: Shopping Cart

```
let cart =  
[  
  "Laptop",  
  "Mobile"  
];  
  
let updatedCart =  
[  
  ...cart,  
  "Headphones"  
];  
  
console.log(updatedCart);
```

Output:

["Laptop","Mobile","Headphones"]

Without Spread Operator

```
let arr1 = [10, 20, 30];
```

```
let arr2 = arr1;
```

```
arr2.push(40);
```

```
console.log(arr1);
```

```
console.log(arr2);
```

Output

```
[10, 20, 30, 40]
```

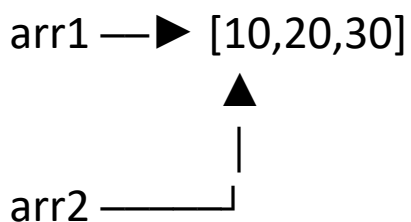
```
[10, 20, 30, 40]
```

Why?

```
let arr2 = arr1;
```

This does **not create a new array**.

Both variables point to the **same memory location**.



When you change arr2, arr1 also changes.

With Spread Operator

```
let arr1 = [10, 20, 30];
```

```
let arr2 = [...arr1];
```

```
arr2.push(40);
```

```
console.log(arr1);
```

```
console.log(arr2);
```

Output

```
[10, 20, 30]
```

```
[10, 20, 30, 40]
```

Now:

arr1 —▶ [10,20,30]

arr2 —▶ [10,20,30,40]

They are completely separate arrays.

JavaScript Output & Input Methods

These are the most commonly used methods for displaying output and taking input in JavaScript:

1. document.write()
2. console.log()
3. prompt()
4. confirm()
5. alert()
6. Related Console Methods
7. Related Document Methods

1. document.write()

Used to display content directly on the web page.

Syntax

```
document.write(data);
```

Example 1

```
<script>

document.write("Hello JavaScript");

</script>
```

Output

Hello JavaScript

Example 2: Multiple Lines

```
<script>

document.write("HTML<br>");
document.write("CSS<br>");
document.write("JavaScript");

</script>
```

Output

HTML

CSS

JavaScript

Example 3: Variables

```
<script>
```

```
let name = "Vicky";
```

```
document.write(name);
```

```
</script>
```

Example 4: Expression

```
<script>
```

```
let a = 10;
```

```
let b = 20;
```

```
document.write(a+b);
```

```
</script>
```

Output

30

2. console.log()

Used to display output in the browser console.

Open Console

F12 → Console Tab

Example 1

```
console.log("Hello World");
```

Output

Hello World

Example 2

```
let name = "Vicky";
```

```
console.log(name);
```

Example 3

```
let a = 10;
```

```
let b = 20;
```

```
console.log(a+b);
```

Output

30

Example 4: Object

```
let student =  
{  
  name:"Vicky",  
  age:25  
};  
  
console.log(student);
```

Related Console Methods

console.error()

Shows error messages.

```
console.error("Something went wrong");
```

Output:

❓ Something went wrong

console.warn()

Shows warning messages.

```
console.warn("Low Battery");
```

Output:

❓ Low Battery

console.table()

Displays array/object in table format.

```
let students =  
[  
  {name:"Aman", age:20},  
  {name:"Rahul", age:21}  
];  
  
console.table(students);
```

console.clear()

Clears console.

```
console.clear();
```

console.time()

Measures execution time.

```
console.time("Loop");
```

```
for(let i=1;i<=1000000;i++)  
{  
}
```

```
console.timeEnd("Loop");
```

Output:

Loop: 5ms

3. prompt()

Used to take input from the user.

Returns a string.

Syntax

```
prompt("Message");
```

Example 1

```
<script>  
  
let name = prompt("Enter Your Name");  
  
document.write(name);  
  
</script>
```

Input

Vicky

Output

Vicky

Example 2: Addition Program

```
<script>  
  
let a = prompt("Enter First Number");
```

```
let b = prompt("Enter Second Number");

let sum = Number(a) + Number(b);

document.write(sum);

</script>
```

Input

10
20

Output

30

Example 3: Student Marks

```
<script>

let marks = prompt("Enter Marks");

if(marks >= 33)
{
    document.write("Pass");
}
else
{
    document.write("Fail");
}
```

</script>

4. confirm()

Used to get Yes/No confirmation from the user.

Returns:

true → OK

false → Cancel

Example 1

<script>

```
let result = confirm("Do you want to continue?");
```

```
document.write(result);
```

</script>

If OK

true

If Cancel

false

Example 2: Delete Record

```
<script>
```

```
let result = confirm("Delete Student Record?");
```

```
if(result)
```

```
{
```

```
    document.write("Record Deleted");
```

```
}
```

```
else
```

```
{
```

```
    document.write("Deletion Cancelled");
```

```
}
```

```
</script>
```

5. alert()

Displays a popup message.

Syntax

```
alert("Message");
```

Example

```
<script>
```

```
alert("Welcome to JavaScript");
```

```
</script>
```

Output

Popup Message

Complete Program Using alert, prompt, confirm

```
<!DOCTYPE html>
<html>
<body>

<script>

alert("Welcome");

let name = prompt("Enter Your Name");

let result = confirm(
"Do you want to display your name?"
);

if(result)
{
    document.write("Welcome " + name);
}
else
{
    document.write("Operation Cancelled");
}

</script>
```

```
</body>  
</html>
```

Related Document Methods

document.writeln()

Adds a line after output.

```
document.writeln("HTML");  
document.writeln("CSS");
```

document.getElementById()

Finds an element by ID.

```
<h1 id="demo"></h1>
```

```
<script>
```

```
document.getElementById("demo").innerHTML =  
"Hello JavaScript";
```

```
</script>
```

innerHTML

Changes HTML content.

```
<div id="box"></div>
```

```
<script>
```

```
document.getElementById("box").innerHTML =  
"<h2>Welcome</h2>";
```

```
</script>
```

innerText

Changes only text.

```
<h1 id="demo"></h1>
```

```
<script>
```

```
document.getElementById("demo").innerText = "Hello World";  
</script>
```

Comparison Table

Method	Purpose	Output Location
document.write()	Display content	Web Page
console.log()	Debugging	Console
alert()	Popup Message	Dialog Box
prompt()	User Input	Dialog Box
confirm()	Yes/No Choice	Dialog Box

Real-World Usage

Development Phase

```
console.log(data);  
console.error(error);  
console.table(users);
```

Used for debugging.

User Input

```
prompt("Enter Name");
```

Used in learning projects and small applications.

Confirmation

```
confirm("Delete Record?");
```

Used before delete, logout, payment, etc.

Type Conversion in JavaScript

Type Conversion means converting a value from one data type to another data type.

For example:

```
let num = "100";
```

```
console.log(typeof num);
```

Output

string

If we convert it to a number:

```
let num = Number("100");
```

```
console.log(typeof num);
```

Output

number

Types of Type Conversion

There are two types:

1. Implicit Type Conversion (Automatic)
2. Explicit Type Conversion (Manual)

1. Implicit Type Conversion

JavaScript automatically converts one data type to another.

Example 1

```
let result = "10" + 5;  
  
console.log(result);  
console.log(typeof result);
```

Output

```
105  
string
```

Why?

```
"10" + 5  
↓  
"10" + "5"  
↓  
"105"
```

JavaScript converts the number into a string.

Example 2

```
let result = "10" - 5;  
  
console.log(result);
```

Output

5

Here JavaScript converts "10" into a number automatically.

Example 3

```
console.log("10" * 2);
```

Output

20

Complete Program (Implicit Conversion)

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<script>
```

```
document.write("10" + 5);
```

```
document.write("<br>");
```

```
document.write("10" - 5);
```

```
document.write("<br>");
```

```
document.write("10" * 5);
```

```
document.write("<br>");
```

```
document.write("10" / 2);
```

```
</script>
```

```
</body>
```

```
</html>
```

Output

105

5

50

5

2. Explicit Type Conversion

When the programmer manually converts one data type into another.

Methods:

- Number()
 - String()
 - Boolean()
-

A. Convert String to Number

Example

```
let x = "100";
```

```
let y = Number(x);
```



```
console.log(y);  
console.log(typeof y);
```

Output

```
100  
number
```

B. Convert Number to String

Example

```
let age = 25;  
  
let result = String(age);  
  
console.log(result);  
console.log(typeof result);
```

Output

```
25  
string
```

C. Convert Value to Boolean

Example 1

```
let x = 1;  
  
console.log(Boolean(x));
```

Output

```
true
```

parseInt()

Used to convert a string into an integer.

Example

```
let num = "123";
```

```
console.log(parseInt(num));
```

Output

123

parseFloat()

Used to convert a string into a decimal number.

Example

```
let num = "123.45";
```

```
console.log(parseFloat(num));
```

Output

123.45
